JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 7 OF 18

Strings and String Problem-Solving Patterns

Unicode-Aware Traversal, Windows, Builders, Parsing, and Text Boundaries

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Treat strings as immutable UTF-16 sequences, choose the right text abstraction, and apply windows, builders, parsing, and frequency patterns safely.

STRINGS | UNICODE | WINDOWS | BUILDERS | PARSING | CHARSETS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `06 – Arrays and Array Patterns`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

String interviews mix algorithm patterns with Java-specific text semantics. This volume makes that boundary explicit so code is both efficient and correct.

Readiness map

Decision	Evidence
START HERE	A substring or contiguous character range matters; Frequency state changes as a window moves; Repeated concatenation creates hidden quadratic work; char, code point, byte, and user-visible character differ.
PREPARE FIRST	Volumes 1-6; Array and sliding-window fundamentals.
MOVE ON WHEN	Choose char, code point, byte, or grapheme abstractions deliberately; Use StringBuilder and window state efficiently; Handle equality, pooling, parsing, and charset boundaries; Explain complexity in terms of code units or code points.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

06 - Arrays and Array Patterns	YOU ARE HERE **07 - Strings and String Patterns**	08 - Hashing and Prefix State

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - String and Unicode Semantics

WHY IT WORKS | First-principles model

An array is an object with a runtime component type and immutable length. Its elements are mutable unless prevented by external design. A variable such as `int[] values` contains a reference, not the elements themselves. Multidimensional array syntax represents arrays whose elements are array references; rows may have different lengths or be null.

A `String` is an immutable sequence of UTF-16 code units. A `char` is one 16-bit code unit. Some Unicode code points use one code unit, others use a surrogate pair, and a visible grapheme can combine several code points. Therefore `length()`, code-point count, and what a user sees are different concepts.

A text block is source syntax for a `String`. It improves multiline readability but does not introduce a different runtime type.

> **Specification boundary:** Java specifies array bounds checks, runtime store checks, `String` behavior, and UTF-16-based APIs. It does not guarantee a particular internal `String` field layout or byte encoding used inside a JVM implementation.

Core terminology

- **Component type:** type of an array's elements.
- **Reified:** runtime retains enough component-type information to check array stores.
- **Covariance:** `Sub[]` is a subtype of `Super[]` when `Sub` is a subtype of `Super`.
- **Aliasing:** multiple references reach the same mutable object.
- **Shallow copy:** copies top-level values or references, not referenced nested objects.
- **Interning:** canonicalizing equal strings in a JVM-managed pool.
- **Code unit:** unit used by an encoding; Java `char` is a UTF-16 code unit.
- **Code point:** Unicode numeric value such as U+1F680.
- **Grapheme cluster:** sequence perceived as one user-visible character.
- **Incidental indentation:** whitespace text blocks remove based on delimiter and content placement.

Detailed mechanics

String identity, value, and pool

String literals and constant string expressions are interned. Runtime-created strings need not share identity.

JAVA

```
String a = "java";
String b = "ja" + "va";        // compile-time constant
String part = "ja";
String c = part + "va";         // runtime concatenation
System.out.println(a == b);      // true
System.out.println(a == c);      // generally false; do not rely on identity
System.out.println(a.equals(c)); // true
```

Use `equals` for exact content and `equalsIgnoreCase` only when its locale-independent comparison matches the domain. For natural-language case conversion, specify a `Locale`. For protocol identifiers, often use `Locale.ROOT`. Unicode normalization is separate from case folding; visually equivalent strings can have different code-point sequences.

Because strings are immutable, operations such as `substring`, `replace`, and `toUpperCase` return a string and do not modify the receiver. `StringBuilder` provides a mutable sequence for single-threaded assembly. `StringBuffer` synchronizes individual operations but rarely solves a higher-level concurrency design.

Modern compilers use `invokedynamic`-based concatenation strategies for many `+` expressions, so simple one-shot concatenation is readable and efficient. Repeated `result += item` in a loop can still create work proportional to all accumulated content; use a builder or joining collector.

Unicode processing

`String.length()` returns UTF-16 code units. `codePointCount` counts Unicode code points. Iterate safely with code-point-aware methods:

JAVA

```
String value = "A\uD83D\uDE80"; // A plus rocket
System.out.println(value.length()); // 3 code units
System.out.println(value.codePointCount(0, value.length())); // 2

value.codePoints().forEach(cp ->
        System.out.printf("U+%04X%n", cp));
```

`codePointAt` combines a valid surrogate pair. If input contains an unpaired surrogate, it returns that surrogate value; Java strings can contain ill-formed UTF-16 sequences. Encoders must decide whether to replace, report, or ignore malformed input.

Unicode escapes such as `\u000A` are processed very early in Java source translation, even in comments. A Unicode escape that becomes a line terminator can change tokenization or make source illegal. Prefer ordinary escapes like `\n` for controls inside literals.

TRACE IT | Worked Java example

This function reverses code points rather than UTF-16 code units and preserves supplementary characters.

JAVA

```java
public class UnicodeReverse {
    static String reverseCodePoints(String input) {
        int[] points = input.codePoints().toArray();
        StringBuilder result = new StringBuilder(input.length());
        for (int i = points.length - 1; i >= 0; i--) {
            result.appendCodePoint(points[i]);
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String input = "A\uD83D\uDE80B";
        System.out.println(reverseCodePoints(input)); // B, rocket, A
    }
}
```

This is code-point correct, not grapheme-cluster correct. Reversing combining marks separately can still produce surprising visible output. User-perceived text segmentation requires a boundary algorithm suitable for the Unicode version and locale.

TRACE IT | Execution or memory walkthrough

The input contains four UTF-16 code units: `A`, a high surrogate, a low surrogate, and `B`. `codePoints()` combines the surrogate pair and emits three `int` values. `toArray` allocates an `int[3]`.

The loop begins at index two and appends `B`. It then calls `appendCodePoint` for the rocket value, which emits two UTF-16 code units, followed by `A`. `toString` creates the immutable result. The original string is unchanged.

No bounds error occurs because the loop starts at `points.length - 1`, continues while nonnegative, and decrements. For an empty input it starts at -1 and performs zero iterations.

Complexity and performance

Array indexing is O(1); traversal and copying are O(n). `System.arraycopy` is still O(n), though JVMs optimize its constant factors. A rectangular `r` by `c` traversal is O(rc); a jagged traversal is proportional to actual elements.

Most string searches and transformations are O(n) in code units, with algorithm-specific exceptions. Concatenating into an immutable accumulator inside a loop can be O(n squared) in total copied content. A pre-sized `StringBuilder` usually makes assembly O(n).

The reverse example is O(n) time and O(n) additional space. A more intricate backward code-point traversal can avoid the `int[]`, but clarity is often worth the allocation unless profiling identifies the path.

> **HotSpot note:** Current HotSpot implementations may store strings compactly using a byte array plus an encoding marker when contents permit. This is not a public `String` contract and should not influence correctness assumptions.

WATCH OUT | Edge cases and common mistakes

- Array assignment aliases; it does not copy elements.
- `clone` on a nested or object array is shallow.
- Covariant array stores can fail at runtime.
- `Arrays.asList(primitiveArray)` creates a one-element list containing that array, not a list of boxed primitives.
- `array.length`, `string.length()`, and `collection.size()` use different syntax.
- `==` compares string references; `equals` compares content.
- Calling `toString` on a null reference fails; `String.valueOf` renders it as `"null"`, which may hide missing data.
- Splitting on a regex metacharacter requires regex escaping, and `split` discards trailing empty strings by default unless a negative limit is used.
- A supplementary code point occupies two `char` values.
- Code-point correctness does not guarantee grapheme or locale correctness.
- Default-charset conversions vary by environment; specify `StandardCharsets.UTF_8` at byte boundaries.
- Text blocks still process escapes and normally include a trailing newline based on delimiter placement.

TRY IT | Exercises

1. Deep-copy a jagged `int[][]` while preserving null rows.
2. Demonstrate an `ArrayStoreException`, then replace the API with a generic collection.
3. Write tests showing trailing-newline and indentation behavior for three text blocks.
4. Count code units, code points, and expected grapheme clusters in strings containing an emoji and a combining mark.
5. Implement UTF-8 encode/decode with a `CharsetEncoder` configured to report malformed input.
6. Replace quadratic loop concatenation with a pre-sized `StringBuilder` and justify the capacity estimate.

SDE-2 CORE CHAPTER

Chapter 2 - Character, Byte, and Charset Boundary

WHY IT WORKS | First-principles model

I/O transfers finite sequences of bytes between a program and an external endpoint. Text is not bytes by itself; a charset maps characters to encoded bytes and back.

```text
Java characters --CharsetEncoder--> bytes --external storage/network
Java characters <--CharsetDecoder-- bytes <--external storage/network
```

Classic I/O exposes sequential streams:

- `InputStream` and `OutputStream` transfer bytes.
- `Reader` and `Writer` transfer characters.
- decorators add buffering, data conversion, compression, or other behavior.

NIO exposes buffers and channels:

```text
channel <-> ByteBuffer <-> application
```

A buffer has three core indexes satisfying:

```text
0 <= position <= limit <= capacity
```

Write into a fresh buffer from `position` toward `limit`. After filling, `flip()` sets `limit` to the amount written and `position` to zero, preparing for reads. After consuming, `clear()` prepares the entire storage for overwrite; `compact()` preserves unread bytes and prepares the remaining suffix for more input.

> **Specification boundary:** Java APIs define resource, buffer, path, channel, and serialization behavior. Operating-system caches, filesystem atomicity support, descriptor limits, direct-buffer allocation, selector implementation, and durability details vary by platform and JDK.

Core terminology

- **Byte stream:** Sequence of raw 8-bit units.
- **Character stream:** Sequence decoded or encoded using a charset.
- **Decorator:** Wrapper that adds behavior while delegating to another stream.
- **Buffering:** Accumulating data to reduce calls and improve transfer efficiency.
- **Channel:** I/O connection supporting buffer-based operations.
- **Path:** Filesystem path representation; it may identify a nonexistent entry.
- **Partial read/write:** Successful operation transferring fewer units than requested.
- **Atomic move:** Move observed as one indivisible namespace update when supported.
- **Durability:** Extent to which data survives process or machine failure.
- **Direct buffer:** Buffer whose storage is intended to support efficient native I/O outside the ordinary Java heap array model.
- **Memory mapping:** Mapping a file region into virtual memory through a buffer-like view.
- **Serialization:** Encoding an object or data model into a transportable representation.

Detailed mechanics

Bytes, characters, and charsets

Use byte APIs for images, compressed data, protocols, hashes, and already encoded content. Use readers and writers for text. Always specify the charset at persistent or network boundaries, commonly `StandardCharsets.UTF_8`. The platform default can differ across environments or be configured, and relying on it makes formats ambiguous.

Unicode characters may require multiple bytes, and a buffer boundary can split one encoded character. `InputStreamReader` and `OutputStreamWriter` maintain encoding state correctly. Converting arbitrary chunks independently with `new String(chunk, UTF_8)` can corrupt a multibyte character split across chunks.

Malformed or unmappable input has a policy. Convenience decoding may replace invalid sequences. When strict validation matters, configure a `CharsetDecoder` with `CodingErrorAction.REPORT`.

WATCH OUT | Edge cases and common mistakes

- Relying on the platform default charset for stored or transmitted text.
- Assuming one read fills an array or one write drains a buffer.
- Forgetting that `read` uses `-1` for end-of-stream.
- Leaking streams returned by `Files.list` or `Files.walk`.
- Calling `readAllBytes` on unbounded input.
- Confusing `normalize`, absolute path, and real path.
- Treating a lexical `startsWith` containment check as symlink-safe authorization.
- Assuming `ATOMIC_MOVE` is supported or silently weakening a required guarantee.
- Believing `flush` or close alone guarantees crash durability.
- Calling `clear()` and assuming sensitive bytes were erased.
- Forgetting `flip()` before reading data just written to a buffer.
- Losing partial protocol frames between nonblocking reads.
- Enabling selector write interest continuously and causing a busy loop.
- Allocating unbounded direct buffers and watching only Java heap metrics.
- Mutating shared storage through a sliced or duplicated buffer alias.
- Deserializing native Java objects from untrusted input.
- Treating `transient` as confidentiality or `serialVersionUID` as safe evolution.

SDE-2 CORE CHAPTER

Chapter 3 - SDE-2 String Problem-Solving Patterns

Why string problems need two contracts

A string problem has an algorithmic contract and a text contract. The algorithmic contract asks whether order, contiguity, frequencies, or a pattern match matters. The text contract asks what one "character" means: a Java UTF-16 code unit, a Unicode code point, a normalized symbol, or a user-perceived grapheme cluster.

Many interview solutions are correct only for lowercase ASCII but silently present themselves as general text algorithms. A strong SDE-2 answer narrows the contract intentionally or pays the cost of a broader representation. This chapter keeps that decision visible while developing palindrome, anagram, frequency, builder, sliding-window, KMP, rolling-hash, Z-algorithm, and parsing patterns.

Learning objectives

After completing this chapter, you should be able to:

- choose among code units, code points, bytes, and grapheme clusters for a stated product contract;
- implement palindrome and anagram checks without splitting surrogate pairs;
- choose an array or map for frequency state based on alphabet guarantees;
- use `StringBuilder` to avoid repeated immutable concatenation;
- derive a non-repeating sliding window and report its unit of measurement;
- build the KMP prefix function and use it for deterministic linear-time search;
- use rolling hash only with exact collision verification;
- explain the Z algorithm as an alternative prefix-matching primitive;
- parse signed ASCII integers with complete validation and overflow detection; and
- discuss normalization, locale, denial-of-service, memory, and API boundaries.

CHOOSE IT | Recognition and decision map

Signal	First pattern	Text contract question
compare from both ends	opposing pointers	code units or code points? normalization?
same symbols in any order	frequency counts	what counts as the same symbol?
longest substring under a uniqueness/count rule	sliding window	what unit indexes the window?
many output fragments	`StringBuilder`	required capacity and escaping?
exact pattern in text, worst-case guarantee	KMP	return code-unit or code-point offsets?
many candidate substrings, probabilistic filter acceptable	rolling hash plus verification	collision and adversarial-input policy?
prefix match length needed at every position	Z algorithm	separator and alphabet safety?
text represents a number or protocol token	explicit parser	grammar, whitespace, signs, radix, overflow?

Do not use a stream pipeline simply because the input is a string. Stateful windows and prefix automata are clearer as loops, and performance-sensitive primitive code points should not be boxed without a reason. Likewise, full I/O and NIO mechanics belong at the system boundary; this chapter focuses on the in-memory DSA decisions after text has been decoded.

Java text model: choose the unit first

`String` is immutable and indexed in UTF-16 code units. `length()` returns code units, and `charAt(i)` returns one 16-bit `char`. A supplementary Unicode code point uses a surrogate pair and therefore occupies two code units. Splitting or reversing those code units independently can corrupt the pair.

`input.codePoints()` combines each well-formed surrogate pair into one Unicode code-point value. An unpaired surrogate is emitted separately as its zero-extended `char` value, which is not a Unicode scalar value. Converting the stream to `int[]` simplifies random access but costs `O(p)` additional space for `p` emitted values. If isolated surrogates can enter the system, the contract must reject, preserve, or replace them explicitly; the exact-sequence helpers below preserve them as distinct emitted values. A forward-only scan can use `codePointAt` and advance by `Character.charCount(cp)` without materializing the array.

A grapheme cluster is closer to a user-perceived character and may contain multiple code points, such as a base letter plus combining marks. Java's core indexing methods do not provide constant-time grapheme indexing. User-facing cursor movement and deletion may need a Unicode segmentation library or a carefully chosen `BreakIterator` policy.

Four defensible contracts are common:

1. **ASCII token:** reject anything outside a named ASCII grammar. This is appropriate for many protocol fields and numeric parsers.

2 **UTF-16 exact sequence:** compare Java code units exactly, matching `String.equals` and `String.indexOf` semantics.

3 **Unicode code-point sequence:** avoid splitting surrogate pairs but do not equate canonically equivalent spellings.

4 **Human-language text:** define normalization, case folding, locale, and grapheme behavior with product owners and test data.

The word "Unicode-aware" is not a complete contract. For example, U+00E9 and `e` followed by U+0301 can display similarly but are different sequences unless normalized. Case conversion can change length and can be locale-sensitive. Use `Locale.ROOT` for locale-neutral identifiers, and a specific user locale for linguistic text.

Pattern 1: palindrome with opposing code-point indexes

For an exact code-point palindrome, convert the string to `int[]`, then compare `left` and `right` while moving inward.

Invariant: every mirrored pair strictly outside `[left, right]` has matched. If the current pair differs, the contract is false. Otherwise the unknown interval shrinks by two. When the pointers meet or cross, every required pair matched.

Time is `O(p)` and auxiliary space is `O(p)` for the simple code-point array. An ASCII or UTF-16-code-unit contract can use `charAt` with `O(1)` extra space. A code-point implementation can also scan from both UTF-16 ends using `codePointAt` and `codePointBefore`, but the index movement is easier to get wrong.

TRACE IT | Dry run

For code points `[r, a, c, e, c, a, r]`:

left/right	comparison	remaining unknown range
0/6	`r == r`	`[1,5]`
1/5	`a == a`	`[2,4]`
2/4	`c == c`	`[3,3]`

The center needs no partner, so the string is a palindrome.

If the requirement says "ignore punctuation and case," define exactly which code points are punctuation and which case-folding policy applies. Skipping with `Character.isLetterOrDigit` and applying `Character.toLowerCase` is one contract, not a universal definition of meaningful text equality.

Pattern 2: anagram and frequency maps

Two strings are exact code-point anagrams when every code point has the same multiplicity. Increment counts for the first string and decrement for the second. Remove zero entries so an empty map at the end proves equality.

Invariant after processing prefixes: for each code point, the map stores its count in the processed prefix of the first string minus its count in the processed prefix of the second. If code-point lengths differ, return false early. Otherwise an empty final map means every difference is zero.

For a guaranteed lowercase English alphabet, an `int[26]` is faster and simpler. For arbitrary code points, a `Map<Integer,Integer>` expresses a sparse alphabet but boxes keys and values. Production text processing at scale may use a primitive collection library after measurement.

TRACE IT | Dry run

For `listen` and `silent`, the first pass builds `{l=1,i=1,s=1,t=1,e=1,n=1}`. The second pass decrements the same six symbols in another order, deleting each at zero. The final map is empty.

Normalization changes the question. If canonically equivalent spellings should compare equal, normalize both strings to the chosen Unicode normalization form before counting. If case should be ignored, case-fold or case-convert under a documented policy before normalization/counting. These transformations can allocate and change length.

Pattern 3: building output without quadratic copying

Because `String` is immutable, repeated concatenation in a loop may copy the accumulated prefix on each iteration. The total copied characters can grow quadratically. `StringBuilder` maintains a growable buffer and usually makes appending `n` total code units amortized `O(n)`.

For run-length encoding, scan equal code points as one run and append the code point followed by its count. The invariant is that the builder encodes exactly the completed runs before the current one. Use `appendCodePoint` when the unit is a code point.

For input code points `a,a,a,b,c,c`, emit `a3b1c2`. Whether a single occurrence should omit `1`, how delimiter ambiguity is escaped, and whether decoding is required are format decisions. A production serialization format should not invent an ambiguous encoding merely to save a few bytes.

Capacity hints can reduce buffer growth when output size is predictable, but avoid untrusted sizes that cause excessive allocation. A `StringBuilder` is not thread-safe; keep it method-local or synchronize ownership. `StringBuffer` synchronization rarely fixes a poorly scoped shared builder design.

Pattern 4: longest substring without repeated code points

Maintain a half-open code-point window `[left, right)`, plus the last index at which each code point appeared. When reading a code point at `right`, a prior occurrence invalidates the window only if that occurrence is at or after `left`. Move `left` to `previous + 1`; never move it backward.

Invariant after processing `right`: `[left, right + 1)` contains no duplicate code point, and `left` is the earliest valid boundary compatible with the most recent occurrence of each symbol. `best` is the maximum valid length seen.

Each right index is processed once, and `left` only advances, so expected time is `O(p)` with a hash map and space is `O(min(p, alphabet))`.

TRACE IT | Dry run

For `a b c a b b` by code-point index:

right/value	previous	new left	current length	best
0/a	none	0	1	1
1/b	none	0	2	2
2/c	none	0	3	3
3/a	0	1	3	3
4/b	1	2	3	3
5/b	4	5	1	3

The answer is 3. Assigning `left = previous + 1` without `max(left, ...)` would move the window backward when a stale occurrence lies before the current left boundary.

If the caller needs the actual substring, code-point indexes cannot be passed directly to `substring`, which expects UTF-16 offsets. Convert with `offsetByCodePoints`, or track code-unit offsets during the scan. State whether the returned length is code points, code units, or graphemes.

Pattern 5: KMP prefix function and search

Naive search may recompare the same text positions after a mismatch. Knuth-Morris-Pratt preprocesses the pattern so it knows the longest pattern prefix that is also a suffix of the portion already matched.

For pattern array `p`, `prefix[i]` is the length of the longest proper prefix of `p[0..i]` that is also its suffix. "Proper" means shorter than the whole range.

To build it, let `matched = prefix[i - 1]`. While `matched > 0` and `p[i] != p[matched]`, fall back to `prefix[matched - 1]`. If the symbols match, increment `matched`. Store it at `prefix[i]`.

The key invariant is that `matched` is a valid border length for the prefix ending at `i - 1`. Fallback follows already computed borders from longest to shorter without moving `i` backward. Across construction and search, each forward move and fallback is amortized linear.

Prefix dry run

For pattern `ababaca`, the prefix array is:

```text
pattern: a b a b a c a
index:   0 1 2 3 4 5 6
prefix:  0 0 1 2 3 0 1
```

At index 5, `c` mismatches the expected `b` after a border of length 3. Fallback to border length 1 still expects `b`, then fallback to zero; `c` does not match `a`, so the entry is zero.

During text search, `matched` is the number of pattern symbols matching a suffix of the processed text. On a full match, report `textIndex - patternLength + 1`, then fall back to allow overlapping matches. For text `zzababacayy`, pattern `ababaca` matches at code-point index 2.

The implementation below searches code-point arrays and returns code-point offsets. Preprocessing is `O(m)`, search is `O(n)`, and extra space is `O(m)`. This deterministic bound is valuable for adversarial text.

Pattern 6: rolling hash with collision verification

Rolling hash computes a fingerprint for the pattern and each same-length text window. A polynomial hash can remove the outgoing symbol and append the incoming symbol in `O(1)`, so candidate discovery is linear after initial hashing.

Hashes are not identities. Different strings can collide. Correct search therefore uses the hash only as a filter and performs an exact comparison when hashes match. The implementation below uses Java's defined 64-bit overflow as arithmetic modulo `2^64`, then calls `regionMatches` before returning.

For text `abracadabra` and pattern `cada`, the windows at starts 0 through 3 have different fingerprints. The start-4 window hashes like the pattern and exact verification confirms `cada`, so the result is 4.

With verification, correctness is deterministic. Expected time is `O(n + m)`, but worst-case time can become `O(nm)` if many windows collide. KMP avoids that worst case. A production system processing hostile input may use KMP, two independent randomized hashes plus verification, or the platform's optimized `String.indexOf`. Never omit verification because a collision "seems unlikely."

This method uses UTF-16 code units and returns a code-unit offset, matching Java substring APIs. That is a deliberate different contract from the code-point KMP helper.

Pattern 7: Z algorithm overview

For a sequence `s`, `z[i]` is the length of the longest prefix of `s` matching the sequence starting at `i`. Maintain a rightmost known matching interval `[left, right)`. If `i` lies inside it, initialize from the mirrored prefix value, capped at `right − i`; then extend by direct comparisons. If the match extends past `right`, update the interval.

Each direct extension advances the right boundary, so total work is linear. To search for a pattern, compute Z values over `pattern + separator + text` and report positions whose Z value equals the pattern length. The separator must not occur in either sequence; using integer arrays allows a sentinel outside the code-point domain.

KMP stores border lengths of pattern prefixes; Z stores prefix-match lengths at every position. Both provide linear search. Z is especially convenient when a task directly asks about prefix agreement, string periods, or matches beginning at every position. KMP is often easier for streaming text because its automaton state carries forward without concatenating the full input.

Pattern 8: strict integer parsing

Parsing is an algorithm problem when the grammar is explicit. Suppose the contract is:

- optional leading `+` or `−`;
- one or more ASCII digits;
- no whitespace, separators, non-ASCII digits, or trailing characters; and
- result must fit in Java `int`.

Accumulate the result as a negative number. `Integer.MIN_VALUE` has magnitude one larger than `Integer.MAX_VALUE`, so negative accumulation represents the complete range without overflowing on the minimum value. Before multiplying by 10 and subtracting a digit, compare against safe limits.

Invariant: after consuming a digit prefix, `result` is the exact negative value of that prefix and remains within the final sign's permitted lower bound. Any rejected character or bound failure terminates with an exception; the parser never returns a partial result.

For `−2147483648`, the negative result can remain `Integer.MIN_VALUE`. For `2147483648`, the positive limit is exceeded and the method rejects it. Inputs `""`, `"+"`, `" 7"`, `"7x"`, and non-ASCII digit forms are invalid under this intentionally narrow grammar.

Production code should normally use `Integer.parseInt` for this exact need. Deriving the parser is valuable in an interview because it exposes grammar, overflow, and partial-parse decisions. Protocol parsers should also cap token length before doing expensive work.

Runnable Java 21 reference implementation

The following class compiles as written. Run checkpoints with `java -ea StringPatternToolkit`.

JAVA (continued 1/7)

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class StringPatternToolkit {
    private static final long HASH_BASE = 911_382_323L;

    private StringPatternToolkit() {
    }

    public static boolean isCodePointPalindrome(String input) {
        requireString(input, "input");
        int[] points = input.codePoints().toArray();
        for (int left = 0, right = points.length - 1; left < right; left++, right--) {
            if (points[left] != points[right]) {
                return false;
            }
        }
        return true;
    }

    public static boolean areCodePointAnagrams(String first, String second) {
        requireString(first, "first");
        requireString(second, "second");
        int[] a = first.codePoints().toArray();
        int[] b = second.codePoints().toArray();
        if (a.length != b.length) {
            return false;
        }
        Map<Integer, Integer> differences = new HashMap<>();
        for (int point : a) {
            differences.merge(point, 1, Integer::sum);
        }
```

JAVA (continued 2/7)

```
        for (int point : b) {
            Integer count = differences.get(point);
            if (count == null) {
                return false;
            }
            if (count == 1) {
                differences.remove(point);
            } else {
                differences.put(point, count - 1);
            }
        }
        return differences.isEmpty();
    }

    public static String runLengthEncodeCodePoints(String input) {
        requireString(input, "input");
        int[] points = input.codePoints().toArray();
        StringBuilder result = new StringBuilder(input.length());
        for (int start = 0; start < points.length; ) {
            int end = start + 1;
            while (end < points.length && points[end] == points[start]) {
                end++;
            }
            result.appendCodePoint(points[start]);
            result.append(end - start);
            start = end;
        }
        return result.toString();
    }

    public static int longestUniqueCodePointSubstring(String input) {
        requireString(input, "input");
        int[] points = input.codePoints().toArray();
        Map<Integer, Integer> lastIndex = new HashMap<>();
```

JAVA (continued 3/7)

```
        int left = 0;
        int best = 0;
        for (int right = 0; right < points.length; right++) {
            Integer previous = lastIndex.put(points[right], right);
            if (previous != null && previous >= left) {
                left = previous + 1;
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    public static int[] prefixFunctionCodePoints(String pattern) {
        requireString(pattern, "pattern");
        return prefixFunction(pattern.codePoints().toArray());
    }

    public static List<Integer> kmpSearchCodePointOffsets(String text, String pattern) {
        requireString(text, "text");
        requireString(pattern, "pattern");
        int[] source = text.codePoints().toArray();
        int[] target = pattern.codePoints().toArray();
        if (target.length == 0) {
            return List.of(0);
        }
        int[] prefix = prefixFunction(target);
        List<Integer> matches = new ArrayList<>();
        int matched = 0;
        for (int i = 0; i < source.length; i++) {
            while (matched > 0 && source[i] != target[matched]) {
                matched = prefix[matched - 1];
            }
            if (source[i] == target[matched]) {
                matched++;
```

JAVA (continued 4/7)

```
            }
            if (matched == target.length) {
                matches.add(i - target.length + 1);
                matched = prefix[matched - 1];
            }
        }
        return matches;
    }

    public static int rollingHashSearchUtf16(String text, String pattern) {
        requireString(text, "text");
        requireString(pattern, "pattern");
        int n = text.length();
        int m = pattern.length();
        if (m == 0) {
            return 0;
        }
        if (m > n) {
            return -1;
        }

        long highestPower = 1;
        for (int i = 1; i < m; i++) {
            highestPower *= HASH_BASE;
        }
        long patternHash = 0;
        long windowHash = 0;
        for (int i = 0; i < m; i++) {
            patternHash = patternHash * HASH_BASE + pattern.charAt(i);
            windowHash = windowHash * HASH_BASE + text.charAt(i);
        }

        for (int start = 0; start <= n - m; start++) {
            if (windowHash == patternHash && text.regionMatches(start, pattern, 0, m)) {
```

JAVA (continued 5/7)

```
                return start;
            }
            if (start < n - m) {
                windowHash -= text.charAt(start) * highestPower;
                windowHash = windowHash * HASH_BASE + text.charAt(start + m);
            }
        }
        return -1;
    }

    public static int parseAsciiInt(String token) {
        requireString(token, "token");
        if (token.isEmpty()) {
            throw new NumberFormatException("empty token");
        }
        int index = 0;
        boolean negative = false;
        char first = token.charAt(0);
        if (first == '-' || first == '+') {
            negative = first == '-';
            index++;
        }
        if (index == token.length()) {
            throw new NumberFormatException("sign without digits");
        }

        int limit = negative ? Integer.MIN_VALUE : -Integer.MAX_VALUE;
        int multiplicationLimit = limit / 10;
        int result = 0;
        while (index < token.length()) {
            char current = token.charAt(index++);
            if (current < '0' || current > '9') {
                throw new NumberFormatException("non-ASCII digit");
            }
```

JAVA (continued 6/7)

```
            int digit = current - '0';
            if (result < multiplicationLimit) {
                throw new NumberFormatException("int overflow");
            }
            result *= 10;
            if (result < limit + digit) {
                throw new NumberFormatException("int overflow");
            }
            result -= digit;
        }
        return negative ? result : -result;
    }

    private static int[] prefixFunction(int[] pattern) {
        int[] prefix = new int[pattern.length];
        for (int i = 1; i < pattern.length; i++) {
            int matched = prefix[i - 1];
            while (matched > 0 && pattern[i] != pattern[matched]) {
                matched = prefix[matched - 1];
            }
            if (pattern[i] == pattern[matched]) {
                matched++;
            }
            prefix[i] = matched;
        }
        return prefix;
    }

    private static void requireString(String value, String name) {
        if (value == null) {
            throw new IllegalArgumentException(name + " must not be null");
        }
    }
```

JAVA (continued 7/7)

```java
    public static void main(String[] args) {
        assert isCodePointPalindrome("racecar");
        assert isCodePointPalindrome("\uD83D\uDE00a\uD83D\uDE00");
        assert !isCodePointPalindrome("abca");

        assert areCodePointAnagrams("listen", "silent");
        assert areCodePointAnagrams("\uD83D\uDE00a", "a\uD83D\uDE00");
        assert !areCodePointAnagrams("ab", "aa");
        assert runLengthEncodeCodePoints("aaabcc").equals("a3b1c2");

        assert longestUniqueCodePointSubstring("abcabb") == 3;
        assert longestUniqueCodePointSubstring("\uD83D\uDE00a\uD83D\uDE00") == 2;

        assert java.util.Arrays.equals(prefixFunctionCodePoints("ababaca"),
                new int[] {0, 0, 1, 2, 3, 0, 1});
        assert kmpSearchCodePointOffsets("zzababacayyababaca", "ababaca")
                .equals(List.of(2, 11));
        assert rollingHashSearchUtf16("abracadabra", "cada") == 4;
        assert rollingHashSearchUtf16("abc", "xyz") == -1;

        assert parseAsciiInt("0") == 0;
        assert parseAsciiInt("+2147483647") == Integer.MAX_VALUE;
        assert parseAsciiInt("-2147483648") == Integer.MIN_VALUE;
        boolean rejected = false;
        try {
            parseAsciiInt("2147483648");
        } catch (NumberFormatException expected) {
            rejected = true;
        }
        assert rejected;
    }
}
```

Complexity and contract table

Pattern	Time	Auxiliary space	Offset/unit returned
code-point palindrome	`O(p)`	`O(p)`	boolean
code-point anagram	expected `O(p + q)`	`O(alphabet)`	boolean
builder run encoding	`O(p)`	`O(output)`	UTF-16 string
unique sliding window	expected `O(p)`	`O(alphabet)`	code-point length
KMP preprocessing/search	`O(m + n)`	`O(m)`	code-point offsets
rolling hash plus verification	expected `O(m + n)`, worst `O(mn)`	`O(1)`	UTF-16 offset
strict integer parser	`O(n)`	`O(1)`	signed `int`

Here `p`, `q`, `m`, and `n` refer to elements in the representation selected by each method. Complexity claims should not say "characters" without defining that unit.

WATCH OUT | Edge cases and common mistakes

1. **Assuming `char` equals a Unicode character.** Supplementary code points use two code units.
2. **Changing normalization silently.** Exact equality and canonical equivalence are different product semantics.
3. **Locale-unspecified case conversion.** Identifier normalization should normally use `Locale.ROOT`; user text may need a user locale and full case folding.
4. **Fixed 26-entry frequency array without validation.** It is correct only for the named alphabet.
5. **Window boundary moves backward.** Use the last occurrence only when it lies inside the current window.
6. **Returning the wrong offset unit.** Java slicing uses UTF-16 offsets; a code-point algorithm may return code-point indexes.
7. **Empty-pattern ambiguity.** Define whether it matches only at zero, at every boundary, or is rejected. The toolkit returns a single match at zero.
8. **KMP fallback off by one.** Fall back to `prefix[matched - 1]`, not `prefix[matched]`.
9. **Hash accepted as equality.** Always verify candidate text exactly.
10. **Quadratic concatenation.** Use a builder for output assembled in a loop.
11. **Partial parse accepted.** A strict parser consumes the entire token or fails.
12. **Overflow checked after it occurs.** Validate before multiply/add, or use a wider exact representation.
13. **Unlimited input.** Text can cause memory or CPU exhaustion even when the algorithm is linear.

SDE-2 production follow-ups

- **Boundary decoding:** decode bytes with an explicit charset and malformed-input policy before these algorithms. Do not reinterpret arbitrary bytes as characters.
- **Normalization placement:** normalize once at a defined ingestion or comparison boundary, not inconsistently in scattered helpers. Preserve original text when display fidelity matters.
- **Security:** visually confusable Unicode identifiers, bidirectional controls, and mixed scripts may require policies beyond algorithmic equality. Authentication identifiers need a security-reviewed canonicalization scheme.
- **Denial of service:** cap token, pattern, and output sizes. Deterministic KMP may be preferable to collision-sensitive hashing for hostile inputs.
- **Memory:** converting a large string to code points allocates another array. A streaming or index-based scan reduces memory but complicates random access.
- **Caching:** cache prefix functions only when patterns repeat and cache cardinality is bounded. User-controlled unbounded keys can exhaust memory.
- **Logging:** avoid logging raw personal or secret text. Record lengths, validation failure categories, and latency.
- **API types:** return a record naming `codePointStart` or `utf16Start` rather than an unexplained integer when offsets cross component boundaries.
- **Libraries:** production exact substring search should normally start with `String.indexOf`, whose implementation can use runtime optimizations. Implement KMP when its semantics or worst-case guarantee is required.
- **Testing:** include supplementary code points, combining sequences, empty strings, lone surrogates if they can enter the system, long repeated prefixes, and integer limits.

TRY IT | Exercises with model checkpoints

Exercise 1: minimum covering window

Find the shortest code-point substring of `text` containing every code point of `target` with multiplicity.

Model checkpoints: build required counts; maintain satisfied versus required distinct keys; expand right and shrink while fully satisfied; record code-point boundaries; convert to UTF-16 offsets only at the API edge; empty-target behavior must be defined.

Exercise 2: Unicode-aware normalized palindrome

Design a palindrome check that ignores punctuation and case and treats canonically equivalent spellings alike.

Model checkpoints: specify normalization form, case policy, locale, and which code points are retained; transformations may expand text; code points still do not equal grapheme clusters; include product/security review rather than claiming one universal answer.

Exercise 3: all KMP matches in a stream

Search chunks of decoded text without concatenating them.

Model checkpoints: retain `matched` between chunks; retain the global code-point count for offsets; the pattern prefix array is reusable; handle a surrogate pair split only at the decoder boundary, not in the KMP state; overlapping matches fall back after reporting.

Exercise 4: implement Z values

Return the Z array for an `int[]` sequence.

Model checkpoints: maintain half-open `[left, right)`; cap mirrored reuse at `right − i`; direct comparison extends only beyond known territory; update the box when `i + z[i] > right`; demonstrate linear aggregate extension.

Exercise 5: group anagrams at scale

Group many lowercase ASCII words, then generalize the design to Unicode text.

Model checkpoints: a 26-count immutable key avoids sorting for ASCII; custom keys need stable equality and hashing; Unicode requires a normalization contract and sparse representation or sorted code points; bound group cardinality and memory.

Exercise 6: quoted CSV field builder

Escape one CSV field by doubling quotes and surrounding the field with quotes when required.

Model checkpoints: define delimiter, quote, CR, and LF rules; scan once and append to a builder; pre-sizing is optional; this is field escaping, not a complete CSV parser; test empty fields and embedded newlines.

Exercise 7: radix parser

Extend strict parsing to bases 2 through 36.

Model checkpoints: validate radix first; map ASCII digits and letters explicitly; reject a digit outside the radix; preserve negative accumulation and pre-operation overflow checks; decide whether prefixes such as `0x` are grammar or invalid text.

Interview answer checklist

- [ ] I defined what one text element means.
- [ ] I stated whether matching is exact, normalized, case-sensitive, or locale-dependent.
- [ ] I chose a frequency array only after proving the alphabet bound.
- [ ] I named the window invariant and offset unit.
- [ ] I can derive KMP fallback rather than quote it as magic.
- [ ] I verify every rolling-hash candidate exactly.
- [ ] I defined empty string and empty pattern behavior.
- [ ] I use a builder for loop-based output construction.
- [ ] My parser defines its entire grammar and rejects partial input.
- [ ] I test supplementary code points, repeated prefixes, and numeric limits.
- [ ] I can explain memory, hostile-input, and normalization trade-offs.

RECAP | Summary

String algorithms become reliable when the text unit is part of the method contract. Opposing pointers prove palindromes; frequency differences prove anagrams; builders make incremental output linear; and sliding windows retain only the state needed for a contiguous constraint. KMP converts repeated prefix knowledge into a deterministic linear search. Rolling hash is an efficient filter only when exact verification preserves correctness. The Z algorithm offers another linear view of prefix agreement. Parsing requires a grammar and checks before arithmetic overflows. These techniques are interview patterns, but the SDE-2 distinction is connecting them to Unicode semantics, offsets, resource limits, and production boundaries.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: reverse, count, parse, and build
- Interview Core: palindrome, anagram, and sliding-window problems
- SDE-2 Follow-up: define Unicode and validation contracts
- Challenge: combine frequency state with minimum/maximum windows

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous volume: 06 - Arrays and Array Problem-Solving Patterns

Series index

Next volume: 08 - Hashing: Maps, Sets, Frequency, and Prefix State

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 7 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.